



**PARMTRICE
CAINE**

Projet PARMTRICAINE®

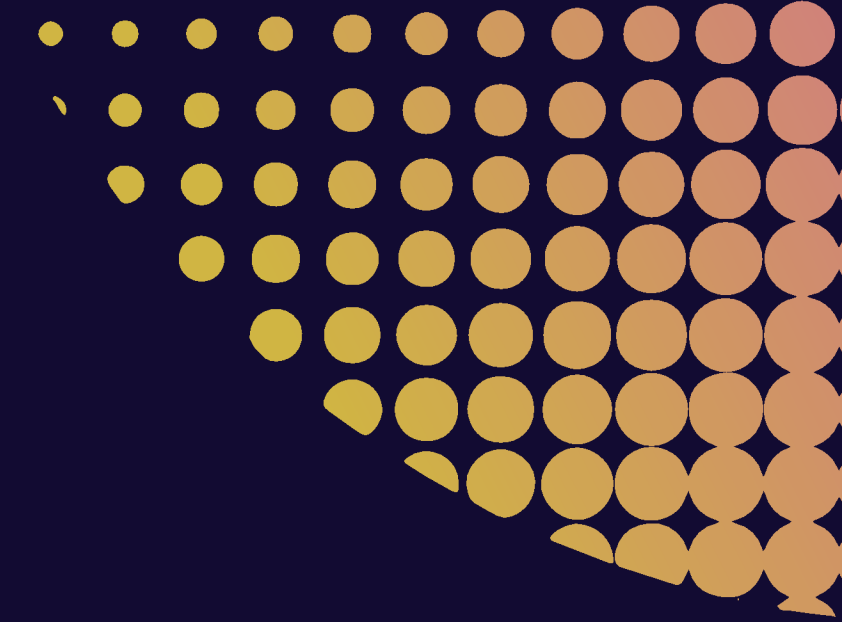
INTRODUCTION

Ce projet a pour objectif de simuler le comportement d'un microprocesseur ARM Cortex-M0 à l'aide du logiciel Logisim. Il consiste à concevoir un processeur 32 bits simplifié, destiné aux systèmes embarqués, en implémentant ses principaux blocs matériels et logiciels ainsi qu'un sous-ensemble du jeu d'instructions ARM, afin de comprendre son fonctionnement interne par la modélisation et la simulation.

SOMMAIRE

- Répartition des tâches au sein du groupe
- Présentation détaillée du projet et des travaux réalisés
- Choix d'implémentation et décisions techniques
- Démo du processeur sur le simulateur Logisim
- Démo de l'assembleur
- Validation et démo des tests
- Limites actuelles et problèmes non résolus
- Conclusion

RÉPARTITION DES TÂCHES



Ulysse V.

- ALU
- Vecteurs de Tests ALU
- Test d'intégration

Thomas B.

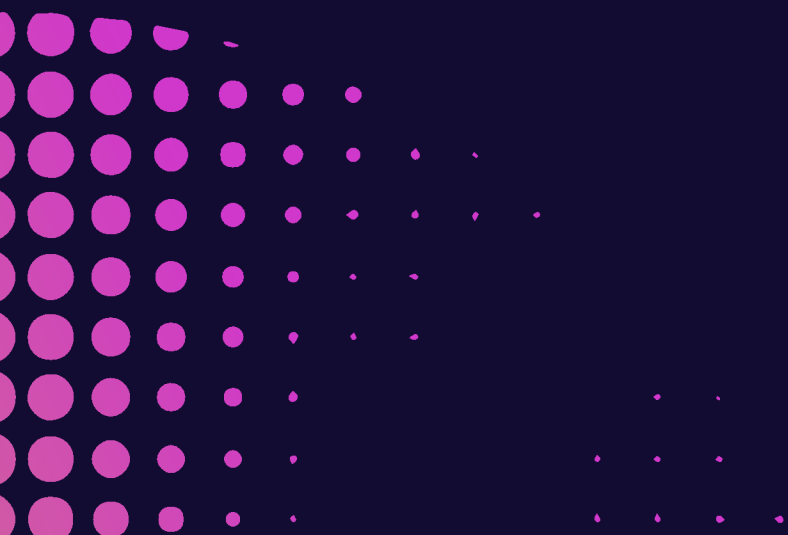
- ALU
- Vecteurs de Tests ALU
- Présentation

Hugo M.

- Contrôleurs
- Tests des contrôleurs
- Test d'intégration
- Tableau des opérations

Thomas D.

- Banc de registre
- Parseur
- Opcode decoder
- CPU

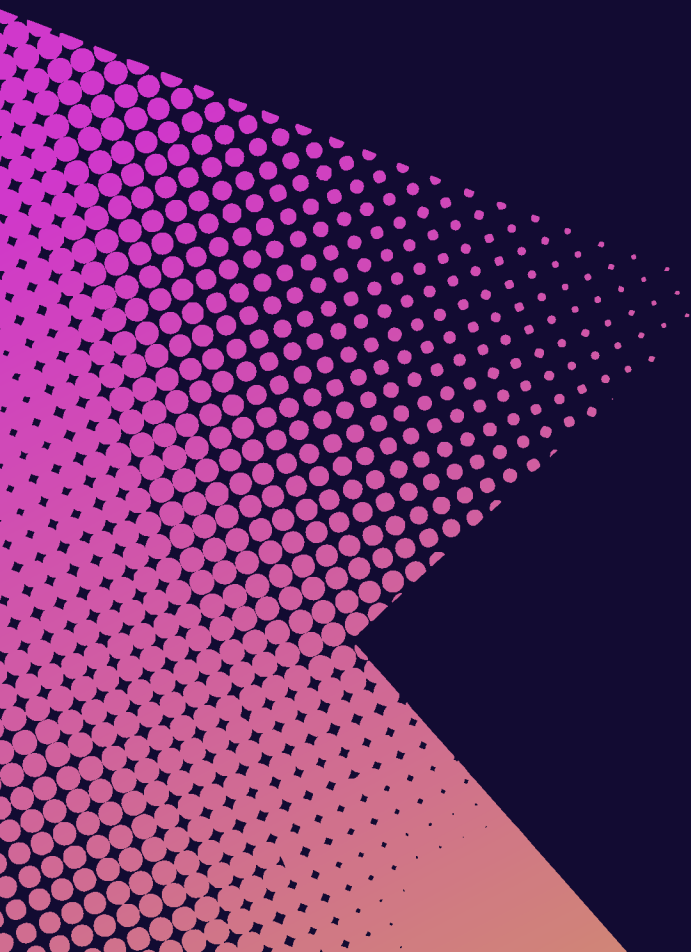


ÉS

données

DATA
(32 bits)

DATA
Memory



CHOIX D'IMPLEMENTATION / DÉCISIONS TECHNIQUES

1. Génération des tests

Un script Python est utilisé pour générer automatiquement un grand nombre de tests, afin d'augmenter la couverture et de révéler d'éventuels cas limites qui auraient pu être oubliés avec des tests écrits manuellement.

2. Choix de Python pour le parseur

Le parseur a été développé en Python plutôt qu'en C pour bénéficier d'une implémentation plus rapide, plus lisible et plus flexible, adaptée au prototypage et aux évolutions fréquentes du jeu d'instructions.

3. Affichage plutôt que tests automatisés pour le parseur

Un affichage détaillé des résultats du parseur a été privilégié afin de faciliter la vérification visuelle et le débogage de la traduction assembleur → binaire, plus adaptés que des tests unitaires à ce stade du projet.

- DÉMO DU PROCESSEUR SUR LE SIMULATEUR LOGISIM
- DÉMO DES COMPOSANTS
- VALIDATION ET DÉMO DES TESTS

AFFICHAGE RÉSULTATS PARSEUR

```
17 -----
18 Traitement : .\code_asm\test_integration\data_processing\5-10_instructions.s
19 succes pour 5-10_instructions.bin
20 -----
21 Traitement : .\code_asm\test_integration\load_store\load_store.s
22 succes pour load_store.bin
23 -----
24 Traitement : .\code_asm\test_integration\miscellaneous\sp.s
25 succes pour sp.bin
26 -----
27 Traitement : .\code_asm\test_integration\shift_add_sub_mov\1-4_instructions.s
28 succes pour 1-4_instructions.bin
29 -----
30 Traitement : .\code_asm\test_integration\shift_add_sub_mov\5-8_instructions.s
31 succes pour 5-8_instructions.bin
32 -----
33 Traitement : .\code_c\calckeyb.s
34 succes pour calckeyb.bin
35 -----
36 Traitement : .\code_c\calculator.s
37 succes pour calculator.bin
38 -----
39 Traitement : .\code_c\simple_add.s
40 succes pour simple_add.bin
41 -----
42 Traitement : .\code_c\testfp.s
43 succes pour testfp.bin
44 -----
45 Traitement : .\code_c\tty.s
46 succes pour tty.bin
47 -----
48 --- Bilan : 15/15 fichiers corrects ---
```

Réduire ↑

LIMITES ACTUELLES ET PROBLÈMES NON RÉSOLUS

Une évolution possible serait de remplacer le parseur Python par une implémentation en C afin d'obtenir un meilleur niveau d'optimisation.

Optimisation de l'ALU encore possible : un multiplexeur pourrait être retiré pour plus d'optimisation.

Erreurs sur le shift : - Erreur de temps en temps

Affichage du résultat Digit: N'est pas fait pour afficher de grands nombres

CONCLUSION

PARMtrice Caine

DERONNE Thomas
MELEIRO Hugo
VANDAMME Ulysse
BOISSON Thomas

